MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Works no. 5 and 6
Operating Systems

Theme: The File System with Your Own Hands (Linux) and Inside the xv6 File System

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 LAB 5. THE FILE SYSTEM WITH YOUR OWN HANDS **4**
1.1 The tree and the file types 4
1.2 Inodes and links 5
1.3 Answers for the inodes and links 6
1.4 Permissions 7
1.5 Answers for the permissions 8
1.6 Disks and the kernel as files 9
1.7 Answers for the disks 10
1.8 Optional tasks 11
2 LAB 6. INSIDE THE XV6 FILE SYSTEM **13**
2.1 What xv6 shows 13
2.2 The program mystat 14
2.3 The limits of the file system 17
2.4 The program find 18
CONCLUSIONS **21**
REFERENCES **22**

INTRODUCTION

Purpose of the work

Both laboratory works study the file system. The fifth one does it from the outside, on Linux, with ordinary commands: the tree of folders and the types of files, the inodes behind the names, hard and symbolic links, permissions and the disks underneath. The sixth one does it from the inside, in xv6, where the same ideas (inodes, directories stored as tables of entries, hard links and a journal) are implemented in a few hundred readable lines of C [2].

The following tasks were set for the work:

a) to identify the types of files and the devices of a Linux system and to measure the size of its main folders;

b) to observe inodes, hard links, symbolic links and the difference between the size of a file and the space it uses;

c) to test the permissions of files and folders, the umask and the special bits;

d) to describe the disk, the mounted file system and the information that the kernel exposes in /proc;

e) to decode what the ls of xv6 prints and to write the program mystat, which shows the fields of struct stat;

f) to measure the limits of the xv6 file system and to write the program find, which walks the tree of folders recursively.

The work was done on the Ubuntu 26.04 virtual machine used in the previous laboratory works, as the normal user wancea, not as root. xv6 was already installed in ~/xv6-riscv; the two new programs were added to it. As allowed by both handouts, the two works are presented in one report, with a separate chapter for each.

1 LAB 5. THE FILE SYSTEM WITH YOUR OWN HANDS

1.1 The tree and the file types

The command `whoami` printed wancea, so the experiments ran without the rights of root. The folder ~/os-lab5 was created for the test files, and `ls /` listed the top of the tree: bin, boot, etc, home, usr, var and the others (Figure 1.1). The command `ls -ld` shows a folder itself instead of its contents, and the first letter of each line is the type of the file.

```
wancea@OS-Lab1-2:~$ whoami
wancea
wancea@OS-Lab1-2:~$ mkdir -p ~/os-lab5 && cd ~/os-lab5
wancea@OS-Lab1-2:~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
wancea@OS-Lab1-2:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 12 16:57 /dev/null
drwxr-xr-x 142 root root 12288 Oct  5 12:43 /etc
-rw-r--r--   1 root root  2772 Sep 29 12:31 /etc/passwd
drwxrwxrwt  15 root root   380 Oct 12 17:01 /tmp
```

Figure 1.1 – The root of the tree and the types of five paths

Table 1.1 – The types of the five paths

Path	Letter	Type	Note
/bin	l	symbolic link	points to usr/bin
/dev/null	c	character device	major 1, minor 3
/etc	d	directory	142 links
/etc/passwd	-	regular file	2772 bytes of text
/tmp	d	directory	the last bit is t, see section 1.3

/bin is not a real folder on this system but a symbolic link to usr/bin, and `file /bin` confirms it (Figure 1.3). Ubuntu keeps all programs in /usr/bin and leaves /bin, /sbin and /lib only as links, for old scripts that use these paths. The same command also showed a detail of Ubuntu 26.04: /usr/bin/ls is itself a symbolic link to ../lib/cargo/bin/coreutils/ls, because this version uses the new implementation of the basic commands written in Rust.

In /dev (Figure 1.2) the first letter c marks the character devices, for example /dev/null (1, 3), /dev/console (5, 1) and /dev/autofs (10, 235). Instead of a size, ls prints the two device numbers: the major number chooses the driver and the minor number the device of that driver. The block devices, found with `ls -l /dev | grep '^b'`, are the virtual disk /dev/sda (8, 0) with its partitions /dev/sda1 and /dev/sda2, the CD drive /dev/sr0 and twenty loop devices that are not shown in the figure.

```
wancea@OS-Lab1-2:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root   root     10, 235 Oct 12 16:57 autofs
drwxr-xr-x  2 root   root         520 Oct 12 16:59 block
drwxr-xr-x  2 root   root          80 Oct 12 16:57 bsg
crw-------  1 root   root     10, 234 Oct 12 16:57 btrfs-control
drwxr-xr-x  3 root   root          60 Oct 12 16:57 bus
lrwxrwxrwx+ 1 root   root           3 Oct 12 16:57 cdrom -> sr0
drwxr-xr-x  2 root   root        3780 Oct 12 17:00 char
crw-------  1 root   tty        5,   1 Oct 12 16:58 console
lrwxrwxrwx  1 root   root          11 Oct 12 16:57 core -> /proc/kcore
wancea@OS-Lab1-2:~/os-lab5$ ls -l /dev | grep '^b' | grep -v loop
brw-rw----  1 root   disk       8,   0 Oct 12 16:57 sda
brw-rw----  1 root   disk       8,   1 Oct 12 16:57 sda1
brw-rw----  1 root   disk       8,   2 Oct 12 16:57 sda2
brw-rw----+ 1 root   cdrom     11,   0 Oct 12 16:59 sr0
```

Figure 1.2 – Character and block devices in /dev

The command `du -sh` adds up the space used by all the files under a folder. /var/log takes 221 MB and /usr 5.2 GB (Figure 1.3), so /usr, which holds all the installed programs and libraries, is about 24 times bigger than the logs.

```
wancea@OS-Lab1-2:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
wancea@OS-Lab1-2:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
221M    /var/log
5.2G    /usr
```

Figure 1.3 – The real types of /usr/bin/ls, /etc/passwd and /bin, and the size of two folders

1.2 Inodes and links

A file with the word hello was created and examined with stat (Figure 1.4). The field Inode: 263824 is the number of the inode, the structure on the disk that holds everything about the file except its name: the type, the size, the owner, the permissions, the times and the addresses of the data blocks. The field Links: 1 is the number of names (directory entries) that point to this inode. The size is 6 bytes, the five letters and the newline.

```
wancea@OS-Lab1-2:~/os-lab5$ echo hello > notes.txt
wancea@OS-Lab1-2:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 263824      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  wancea)   Gid: ( 1000/  wancea)
Access: 2026-10-12 17:05:40.130228902 +0000
Modify: 2026-10-12 17:05:40.130228902 +0000
Change: 2026-10-12 17:05:40.130228902 +0000
 Birth: 2026-10-12 17:05:40.130228902 +0000
wancea@OS-Lab1-2:~/os-lab5$ ls -li notes.txt
263824 -rw-rw-r-- 1 wancea wancea 6 Oct 12 17:05 notes.txt
```

Figure 1.4 – stat and ls -li of a new file

The command `ln notes.txt hard.txt` created a second name for the same inode: both names showed the inode 263824 and the link count 2 (Figure 1.5). After `rm notes.txt` the file was still readable through hard.txt, and its link count went back to 1.

```
wancea@OS-Lab1-2:~/os-lab5$ ln notes.txt hard.txt
wancea@OS-Lab1-2:~/os-lab5$ ls -li notes.txt hard.txt
263824 -rw-rw-r-- 2 wancea wancea 6 Oct 12 17:05 hard.txt
263824 -rw-rw-r-- 2 wancea wancea 6 Oct 12 17:05 notes.txt
wancea@OS-Lab1-2:~/os-lab5$ rm notes.txt
wancea@OS-Lab1-2:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
263824 -rw-rw-r-- 1 wancea wancea 6 Oct 12 17:05 hard.txt
```

Figure 1.5 – A hard link survives the removal of the original name

A symbolic link is a different kind of object (Figure 1.6). soft.txt received its own inode, 263607, the type l and the size 8. Reading it worked while hard.txt existed, but after `rm hard.txt` the command `cat soft.txt` failed with No such file or directory, and ls showed the link in red as a broken link.

```
wancea@OS-Lab1-2:~/os-lab5$ ln -s hard.txt soft.txt
wancea@OS-Lab1-2:~/os-lab5$ ls -li hard.txt soft.txt
263824 -rw-rw-r-- 1 wancea wancea 6 Oct 12 17:05 hard.txt
263607 lrwxrwxrwx 1 wancea wancea 8 Oct 12 17:05 soft.txt -> hard.txt
wancea@OS-Lab1-2:~/os-lab5$ cat soft.txt
hello
wancea@OS-Lab1-2:~/os-lab5$ rm hard.txt
wancea@OS-Lab1-2:~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 wancea wancea 8 Oct 12 17:05 soft.txt -> hard.txt
```

Figure 1.6 – A symbolic link breaks when its target is removed

The new folder d had the link count 2, and after creating d/x and d/y the count became 4 (Figure 1.7). A file of 2 bytes, one.txt, showed 2 bytes in ls but 4.0K in du, while the file big.img made with `truncate -s 1G` showed 1.0G in ls and 0 in du.

```
wancea@OS-Lab1-2:~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 wancea wancea 4096 Oct 12 17:06 d
wancea@OS-Lab1-2:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 wancea wancea 4096 Oct 12 17:06 d
wancea@OS-Lab1-2:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 wancea wancea 2 Oct 12 17:06 one.txt
4.0K    one.txt
wancea@OS-Lab1-2:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 wancea wancea 1.0G Oct 12 17:06 big.img
0       big.img
wancea@OS-Lab1-2:~/os-lab5$ rm big.img
```

Figure 1.7 – Link counts of a folder, and size against space

1.3 Answers for the inodes and links

1. In the output of stat the inode number is the field Inode (263824), and the number of names of the file is the field Links (1, and 2 after ln).

2. A name is only an entry in a folder that holds a file name and an inode number. rm calls the system call unlink, which removes one such entry and decreases the link count of the inode. The inode and its data blocks are freed only when the count reaches 0 and no process has the file open. After `rm notes.txt` the inode still had one name, hard.txt, so nothing was lost.

3. A symbolic link does not point to the inode but stores a path, here the text hard.txt, and the kernel follows that path every time the link is opened. After hard.txt was removed the path led nowhere, so the link exists but its target does not. Its size is 8 bytes because the text hard.txt has exactly 8 characters.

4. The 2 links of a new folder are its entry d in ~/os-lab5 and its own entry . inside d. Every subfolder contains the entry .., which points back to its parent, so d/x and d/y added one link each, which gives 4. In general a folder has 2 links plus one for each subfolder.

5. The disk is allocated in blocks of 4 KB (IO Block: 4096 in Figure 1.4), and even the smallest file that has any data needs a whole block, so 2 bytes use 4.0K. truncate only writes the new size into the inode and allocates no blocks. Such a file is called sparse: the missing blocks are holes that read as zeros and take no space until something is written into them.

1.4 Permissions

A file with the mode 600 could be read by its owner, with the mode 000 cat failed with Permission denied, and after `chmod u+r` it worked again (Figure 1.8). The umask is 0002, and a new file and a new folder received the modes -rw-rw-r-- and drwxrwxr-x.

```
wancea@OS-Lab1-2:~/os-lab5$ echo secret > s.txt
wancea@OS-Lab1-2:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 wancea wancea 7 Oct 12 17:06 s.txt
wancea@OS-Lab1-2:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
wancea@OS-Lab1-2:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
wancea@OS-Lab1-2:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt n
ewdir
0002
-rw-rw-r-- 1 wancea wancea    0 Oct 12 17:06 new.txt
drwxrwxr-x 2 wancea wancea 4096 Oct 12 17:06 newdir
```

Figure 1.8 – File permissions and the umask

For the folder box, read permission alone (u=r) allowed ls to print the name f, but cat box/f failed. Execute permission alone (u=x) gave the opposite result: ls box failed with cannot open directory, while cat box/f printed hi (Figure 1.9).

```
wancea@OS-Lab1-2:~/os-lab5$ mkdir box ; echo hi > box/f
wancea@OS-Lab1-2:~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f
cat: box/f: Permission denied
wancea@OS-Lab1-2:~/os-lab5$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
wancea@OS-Lab1-2:~/os-lab5$ chmod 755 box
```

Figure 1.9 – The read and execute bits on a folder

The program /usr/bin/passwd has the mode -rwsr-xr-x and belongs to root, /tmp has the mode drwxrwxrwt, and /etc/shadow, -rw-r----- with the owner root and the group shadow, cannot be read by a normal user (Figure 1.10).

```
wancea@OS-Lab1-2:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 15 root root 780 Oct 12 17:05 /tmp
wancea@OS-Lab1-2:~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
wancea@OS-Lab1-2:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1312 Oct  1 08:42 /etc/shadow
```

Figure 1.10 – The setuid and sticky bits, and /etc/shadow

1.5 Answers for the permissions

1. Each octal digit is the sum of r = 4, w = 2 and x = 1, for the owner, the group and the others. Table 1.2 shows the four modes.

Table 1.2 – Four common modes

Octal	rwx	Typical use
600	rw-------	a private file, such as an SSH private key, which only the owner may read
644	rw-r--r--	an ordinary file that everyone may read but only the owner may change, such as /etc/passwd
755	rwxr-xr-x	a program or a public folder, such as /usr/bin/ls or /etc
700	rwx------	a private folder or script, such as ~/.ssh

2. The kernel checks only one class of bits: if the user is the owner, it uses the owner bits and does not look further. With 000 the owner bits give no read permission, so access is denied. Ownership gives only the right to change the mode, which is why `chmod u+r` worked without root.

3. The umask 0002 lists the bits that are removed from every new file. Files are created with 666 (programs do not ask for x by default) and folders with 777, so the result is 664 (rw-rw-r--) for new.txt and 775 (rwxrwxr-x) for newdir. The mask takes away only the write right of the others.

4. A folder is a table of names and inode numbers. The r bit allows reading this table, so ls could list the name f, but without x the kernel does not let the process pass through the folder to the inode of f, so it could not open the file. The x bit allows exactly this passage (searching the folder for a known name), so cat box/f worked, but without r the list of names could not be read.

5. The s in the owner bits of /usr/bin/passwd is the setuid bit: the program runs with the user ID of its owner, root, whoever starts it. passwd needs it because it changes /etc/shadow, which holds the password hashes and is not even readable for normal users. The t on /tmp is the sticky bit: everyone may create files in /tmp, but a file can be removed or renamed only by its owner (or the owner of the folder, or root), so users cannot delete each other's files.

1.6 Disks and the kernel as files

lsblk lists the block devices as a tree (Figure 1.11). The virtual disk sda has 25 GB and two partitions: sda1 (1 GB) mounted at /boot/efi and sda2 (23.9 GB) mounted at /. sr0 is the CD drive with the Guest Additions of VirtualBox, and loop0 to loop18 are the snap packages, each a compressed image mounted under /snap.

```
wancea@OS-Lab1-2:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/2124
loop2    7:2    0  66.8M  1 loop /snap/core24/1643
loop3    7:3    0  14.2M  1 loop /snap/desktop-security-center/188
loop4    7:4    0  14.2M  1 loop /snap/desktop-security-center/206
loop5    7:5    0 614.5M  1 loop /snap/gnome-46-2404/164
loop6    7:6    0 260.3M  1 loop /snap/firefox/8763
loop7    7:7    0  16.5M  1 loop /snap/firmware-updater/226
loop8    7:8    0  12.7M  1 loop /snap/firmware-updater/258
loop9    7:9    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop10   7:10   0   1.5M  1 loop /snap/hwctl/123
loop11   7:11   0  18.4M  1 loop /snap/prompting-client/228
loop12   7:12   0  18.8M  1 loop /snap/prompting-client/222
loop13   7:13   0   1.5M  1 loop /snap/hwctl/131
loop14   7:14   0   402M  1 loop /snap/mesa-2404/1839
loop15   7:15   0  50.1M  1 loop /snap/snapd/27710
loop16   7:16   0  11.8M  1 loop /snap/snap-store/1390
loop17   7:17   0  44.7M  1 loop /snap/snapd/28254
loop18   7:18   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0    25G  0 disk
├─sda1   8:1    0     1G  0 part /boot/efi
└─sda2   8:2    0  23.9G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/wancea/VBox_GAs_7.2.20
```

Figure 1.11 – The block devices

df -h shows the size and the free space of every mounted file system, and `mount | grep " / "` the type of the root one: ext4, mounted read and write (Figure 1.12). The lines with tmpfs, efivarfs and none are virtual file systems that live in memory and have no disk behind them.

```
wancea@OS-Lab1-2:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           425M  2.1M  423M   1% /run
/dev/sda2        24G  8.0G   15G  36% /
tmpfs           1.1G     0  1.1G   0% /dev/shm
efivarfs        256K  133K  119K  53% /sys/firmware/efi/efivars
tmpfs           1.1G  176K  1.1G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda1       1.1G  6.4M  1.1G   1% /boot/efi
tmpfs           213M   84K  213M   1% /run/user/1000
/dev/sr0         51M   51M     0 100% /run/media/wancea/VBox_GAs_7.2.20
wancea@OS-Lab1-2:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
```

Figure 1.12 – The mounted file systems

Finally the files in /proc were read (Figure 1.13). They are not stored anywhere: the kernel creates their contents at the moment they are read.

```
wancea@OS-Lab1-2:~/os-lab5$ grep -c processor /proc/cpuinfo
4
wancea@OS-Lab1-2:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        2175032 kB
MemFree:          252524 kB
MemAvailable:    1009312 kB
wancea@OS-Lab1-2:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   8305
Ngid:   0
wancea@OS-Lab1-2:~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 wancea wancea 64 Oct 12 17:08 0 -> /dev/pts/0
lrwx------ 1 wancea wancea 64 Oct 12 17:08 1 -> /dev/pts/0
lrwx------ 1 wancea wancea 64 Oct 12 17:08 2 -> /dev/pts/0
lr-x------ 1 wancea wancea 64 Oct 12 17:08 3 -> /proc/8308/fd
```

Figure 1.13 – Information from /proc

1.7 Answers for the disks

1. The disk sda (25 GB) is divided into sda1, 1 GB, the EFI system partition mounted at /boot/efi, and sda2, 23.9 GB, mounted at / with the file system ext4. The CD drive sr0 and the loop devices of the snaps are not partitions of the disk.

2. The root file system is 36 % full: 8.0 GB used and 15 GB free out of 24 GB. The used and the free space together are smaller than the size because ext4 keeps about 5 % of the blocks in reserve for root.

3. According to /proc the machine has 4 processors (`grep -c processor /proc/cpuinfo`) and MemTotal of 2175032 kB, about 2.1 GB. This is the virtual machine, not the laptop itself: for this session it was given 4 virtual processors and 2.5 GB of memory, and MemTotal is smaller than that because the firmware and the kernel keep part of the memory for themselves.

4. /proc/self/status showed Name: head. /proc/self is a link to the folder of the process that reads it, and the file was read by head. In the same way `ls -l /proc/self/fd` showed the descriptors of ls: 0, 1 and 2 lead to the terminal /dev/pts/0, and 3 is the folder /proc/8308/fd that ls itself had opened to list it.

1.8 Optional tasks

Two of the optional tasks were done. For task (a) the first bytes of two files were shown with xxd (Figure 1.14). one.txt contains only the bytes 78 0a, the letter x and a newline, and file calls it ASCII text. After `gzip -c /etc/passwd > notes.txt` the file has the name of a text file, but file recognised gzip compressed data, with the original name passwd and its modification time. The reason is in the first bytes: 1f 8b is the magic number of gzip, 08 is the compression method deflate and the next 08 is a flag that says the original name is stored. The four bytes 92 af bb 6a are the modification time in little endian order, 0x6abbaf92 = 1790685074 seconds since 1970, which is 29 September 2026 12:31:14, the time shown by ls for /etc/passwd. They are followed by 00, 03 (Unix) and the name passwd. file reads these bytes and ignores the extension of the name.

```
wancea@OS-Lab1-2:~/os-lab5$ file one.txt ; xxd one.txt | head -1
one.txt: ASCII text
00000000: 780a                                     x.
wancea@OS-Lab1-2:~/os-lab5$ gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd
notes.txt | head -1
notes.txt: gzip compressed data, was "passwd", last modified: Tue Sep 29 12:31:14 2
026, from Unix, original size modulo 2^32 2772
00000000: 1f8b 0808 92af bb6a 0003 7061 7373 7764  .......j..passwd
```

Figure 1.14 – Magic numbers

For task (d) an empty image of 64 MB was formatted with mkfs.ext4, mounted through a loop device with sudo and examined with df (Figure 1.15). mkfs created 16384 blocks of 4 KB and 16384 inodes, one inode for every 4 KB of space, and a journal of 1024 blocks. After mounting, df -h shows only 56 MB, because the inode table (16384 inodes of 256 bytes, 4 MB) and the journal (4 MB) take space before any file is written. df -i shows 12 inodes already in use: the first ten are reserved for the file system itself, for example inode 2 is the root folder and inode 8 the journal, inode 11 is the folder lost+found, and inode 12, as debugfs showed, is the orphan file, a hidden file of 128 KB that newer versions of mkfs.ext4 create to remember files deleted while they are still open.

```
wancea@OS-Lab1-2:~/os-lab5$ truncate -s 64M disk.img ; mkfs.ext4 disk.img
mke2fs 1.47.2 (1-Jan-2025)
Discarding device blocks: done
Creating filesystem with 16384 4k blocks and 16384 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

wancea@OS-Lab1-2:~/os-lab5$ sudo mount -o loop disk.img /mnt
[sudo: authenticate] Password:
wancea@OS-Lab1-2:~/os-lab5$ df -h /mnt ; df -i /mnt ; sudo umount /mnt
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop19      56M  152K   52M   1% /mnt
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop19     16384    12 16372    1% /mnt
```

Figure 1.15 – A file system inside a file

2 LAB 6. INSIDE THE XV6 FILE SYSTEM

2.1 What xv6 shows

The type of a file in xv6 is a number defined in kernel/stat.h (Figure 2.1), next to struct stat, which the system call stat fills: the device, the inode number, the type, the number of links and the size.

```
wancea@OS-Lab1-2:~/xv6-riscv$ cat kernel/stat.h
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device

struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

Figure 2.1 – kernel/stat.h

On a fresh disk ls printed one line for each entry of the root folder: the name, the type, the inode number and the size in bytes (Figure 2.2). Then the folder a was created and listed.

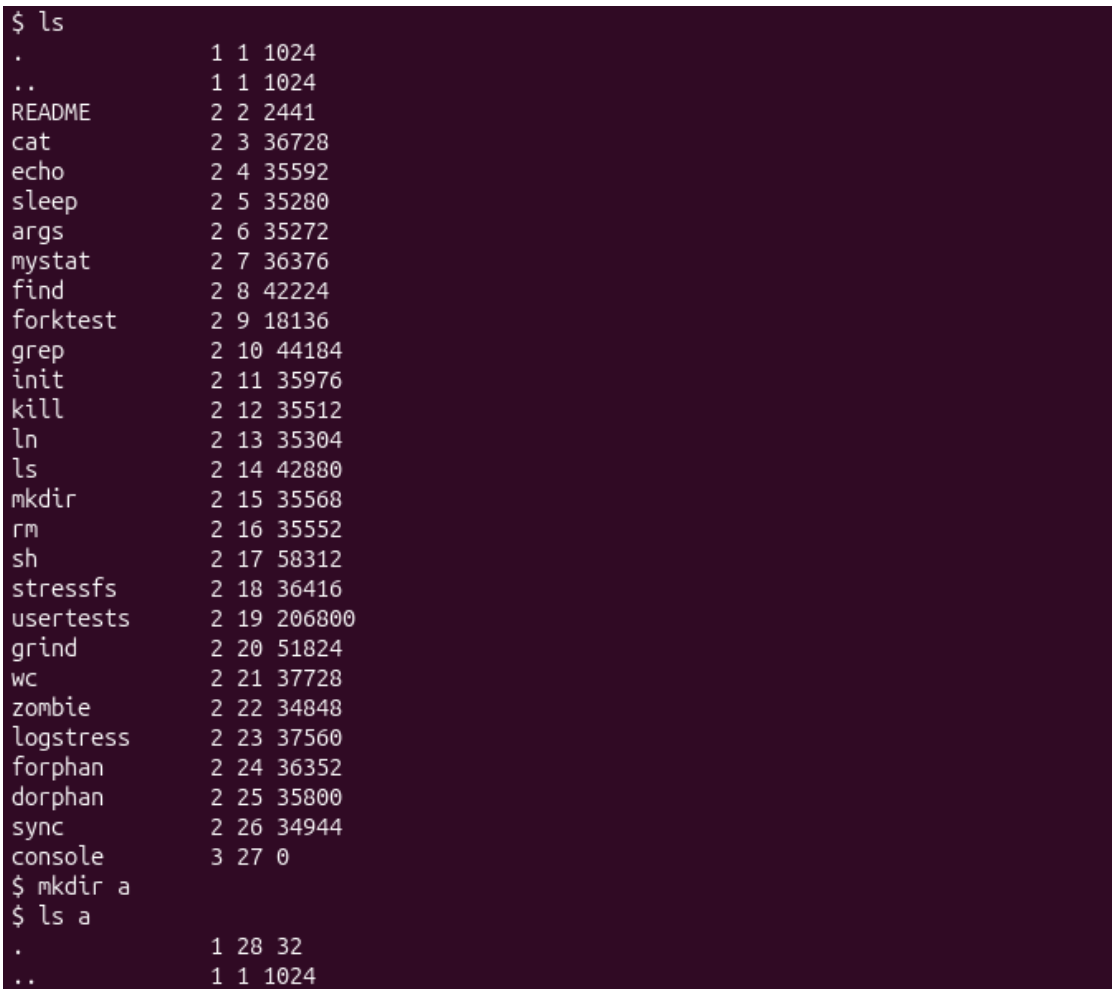


Figure 2.2 – ls in xv6 and a new empty folder

1. Type 1 is T_DIR, a directory; type 2 is T_FILE, an ordinary file; type 3 is T_DEVICE, a device. In Figure 2.2 the folders . and .. have type 1, the programs and README type 2 and console type 3.

2. The root folder has the inode number 1 (ROOTINO in kernel/fs.h). The entry .. of a folder points to its parent, but the root has no parent, so mkfs makes .. of the root point to the root itself, and both . and .. show the inode 1.

3. A directory entry, struct dirent in kernel/fs.h, is a ushort inum of 2 bytes followed by a name of DIRSIZ = 14 bytes, 16 bytes in total. The empty folder a has the size 32 = 2 × 16, because it contains two entries: . with its own inode 28 and .. with the inode 1 of the root.

2.2 The program mystat

The program user/mystat.c (Listing 2.1) calls stat for every argument. stat is a library function in user/ulib.c that opens the file, calls fstat on the descriptor and closes it again. typename converts the type number into a word with a switch over the constants of kernel/stat.h. If stat fails, a message goes to the descriptor 2, the error output, and the loop continues with the next argument. The size

field is a 64 bit number, so it is cast to int for printf, and the program ends with exit(0), because an xv6 program cannot simply return from main. The line `$U/_mystat\` was added to UPROGS in the Makefile.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t)
{
  switch(t){
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat path...\n");
    exit(1);
  }
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Listing 2.1 – user/mystat.c

The output for README, . and console matches the example of the handout except for two numbers (Figure 2.3). console has the inode 27 instead of 25, because it is created by init after all the programs, and this disk holds more programs than the one of the handout (sleep, args, mystat and find were added). The root folder has 2 links instead of 1 because of the folder a created in Part 1. When xv6 creates a folder, the function create in kernel/sysfile.c increments the link count of the parent for the entry .. of the child (line 311), while the entry . is not counted at all (line 301, with the comment "avoid cyclic ref count"). This is why a new folder in xv6 has 1 link and not 2 as on Linux.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.3 – mystat before and after ln and rm

The rest of Figure 2.3 shows a hard link at work. After `ln README r2` both names show the same inode 2 and the link count 2. After `rm README` the name r2 still shows the inode 2 with the size 2441 and the link count 1, and `cat r2 | wc` counted 48 lines, 336 words and 2441 bytes, the whole text of the former README.

```
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n -A 8 "^struct dinode" kernel/fs.h
32:struct dinode {
33-  short type;               // File type
34-  short major;              // Major device number (T_DEVICE only)
35-  short minor;              // Minor device number (T_DEVICE only)
36-  short nlink;              // Number of links to inode in file system
37-  uint size;                // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
40-
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n -A 3 "^struct dirent" kernel/fs.h
58:struct dirent {
59-  ushort inum;
60-  char name[DIRSIZ] __attribute__((nonstring));
61-};
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "nlink--" kernel/sysfile.c ; grep -n "itrunc(
ip)" kernel/fs.c
178:  ip->nlink--;
239:    dp->nlink--;
244:  ip->nlink--;
365:    itrunc(ip); // free the data blocks (type stays nonzero on disk)
```

Figure 2.4 – struct dinode, struct dirent and the lines that decrement nlink

1. After ln the inode number (2) and the size (2441) stayed the same and only the link count changed from 1 to 2, under both names. After rm the name README disappeared, the count went back to 1, and the inode, the size and the data did not change.

2. In kernel/sysfile.c the function sys_unlink decrements the link count in line 244, `ip->nlink--;`, and writes the inode back to the disk with iupdate (Figure 2.4). The data blocks are not freed there. They are freed in iput in kernel/fs.c, line 365, `itrunc(ip)`, which runs when the last reference to the inode in memory is dropped and nlink is 0. So a file that is still open stays readable after its last name is removed, as on Linux.

3. struct dinode, the inode on the disk, has only six fields: type, major, minor (the device numbers, used only for devices), nlink, size and addrs, the addresses of the 12 direct blocks and of one indirect block. Every field is 2 or 4 bytes, so one inode takes 64 bytes and 16 inodes fit in a block. There is no owner, no mode and no time, which is why xv6 has no permissions, and there is no type for a symbolic link.

2.3 The limits of the file system

The constants were found with grep in kernel/fs.h, and the disk image was rebuilt from scratch with `make clean ; make fs.img`, which prints the layout of the disk in the line nmeta (Figure 2.5).

```
wancea@OS-Lab1-2:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRE
CT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
wancea@OS-Lab1-2:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_sleep user/_args user/_mystat user/_find user/_forktest
user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh us
er/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user
/_forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
 total 2000
```

Figure 2.5 – The constants of fs.h and the layout of the disk

After `mkdir averyveryverylongname`, ls showed the new folder as averyveryveryl (Figure 2.6). It also took the third place in the listing, the slot of the entry README removed in Part 2: the entry had been freed by writing 0 into its inum, and the next new entry reused it.

```
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
averyveryveryl 1 29 32
cat            2 3 36728
r2             2 2 2441
$ ls averyveryverylongname
.              1 29 32
..             1 1 1024
$ mystat averyveryverylongname averyveryveryl
averyveryverylongname: directory, inode 29, links 1, size 32 bytes
averyveryveryl: directory, inode 29, links 1, size 32 bytes
```

Figure 2.6 – A name longer than DIRSIZ (middle of the listing left out)

1. ls shows averyveryveryl, the first 14 characters of the name, because a directory entry has room for only DIRSIZ = 14 characters and the kernel cuts longer names without an error. The lookup

compares only the first DIRSIZ characters as well, so the folder can be opened with the full name or the cut one, and mystat finds the same inode 29 under both names.

2. One inode addresses NDIRECT = 12 blocks directly and one indirect block. The indirect block has BSIZE = 1024 bytes and holds block numbers of 4 bytes, so NINDIRECT = 1024 / 4 = 256 more blocks. The largest file has (12 + 256) × 1024 = 268 × 1024 = 274 432 bytes, that is 268 KB.

3. The line `nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000` describes the whole disk of 2000 blocks (Table 2.1).

Table 2.1 – The layout of the xv6 disk

Part	Blocks	Why
boot block	1	block 0, not used by xv6
superblock	1	the sizes and the positions of the other parts
log	31	one header block and LOGBLOCKS = 30 data blocks
inodes	13	NINODES = 200 inodes, 16 per block, 200 / 16 + 1
bitmap	1	one bit for each of the 2000 blocks, 8192 bits fit in one block
data	1953	2000 − 47 blocks for the files and folders

2.4 The program find

The program find (Listing 2.2) is built on the model of user/ls.c, as the handout suggests. The function find opens the folder, checks with fstat that it really is a folder and reads it one struct dirent at a time. Free slots, whose inum is 0, are skipped, and so are the entries . and .., because entering them would make the walk go in circles or back up the tree. For every other entry the full path is built in a buffer of 512 bytes: the path of the folder, a slash and the name, copied with memmove and ended with a 0, because a name of exactly 14 characters has no 0 of its own. stat then gives the type of the entry. If the name is equal to the searched one (compared with strcmp), the path is printed, and if the entry is a folder, find calls itself on it. The folder is closed before the function returns, so only one descriptor per level of depth is open at a time, well below the limit of 16 open files per process. An optional third argument -n makes the program print the inode number next to each path, which is the stretch task of the handout. Both programs were added to UPROGS after the line of args (lines 137 and 138 of the Makefile).

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

int showino;   // set by the optional third argument -n

// print every path under dir whose last component is name
void
find(char *dir, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(dir, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", dir);
    return;
  }
  if(fstat(fd, &st) < 0 || st.type != T_DIR){
    fprintf(2, "find: %s is not a directory\n", dir);
    close(fd);
    return;
  }
  if(strlen(dir) + 1 + DIRSIZ + 1 > sizeof buf){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }
  strcpy(buf, dir);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)                    // free slot
      continue;
    memmove(p, de.name, DIRSIZ);        // a 14-char name has no 0
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;                         // never walk back up or in place
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(showino)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc == 4 && strcmp(argv[3], "-n") == 0)
```

```
    showino = 1;
  else if(argc != 3){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
}
```

Listing 2.2 – user/find.c

The test was run on a fresh disk, built again with make clean. On this virtual machine, which has only two virtual processors, xv6 with its default three harts booted too slowly, so it was started with one hart: `make qemu CPUS=1`. The program itself does not depend on the number of harts. After creating the folders and files of the handout, `find . r2` printed the three paths ./a/b/r2, ./a/r2 and ./r2, and `find a b` printed a/b (Figure 2.7). The order is the order of the entries in each folder: a comes before r2 in the root because it was created first, and the walk enters a before it reaches r2.



Figure 2.7 – find on the test tree of the handout

With the argument -n the three paths received the inode numbers 30, 31 and 3 (Figure 2.8). mystat confirms the result: cat and r2 both have the inode 3 and the link count 2, so ./r2 is the same file as cat, created by `ln cat r2`, while a/r2 and a/b/r2 are two different files of 2 bytes. The last two commands show the error handling: a missing folder gives `find: cannot open nosuch`, and a wrong number of arguments prints the usage line.

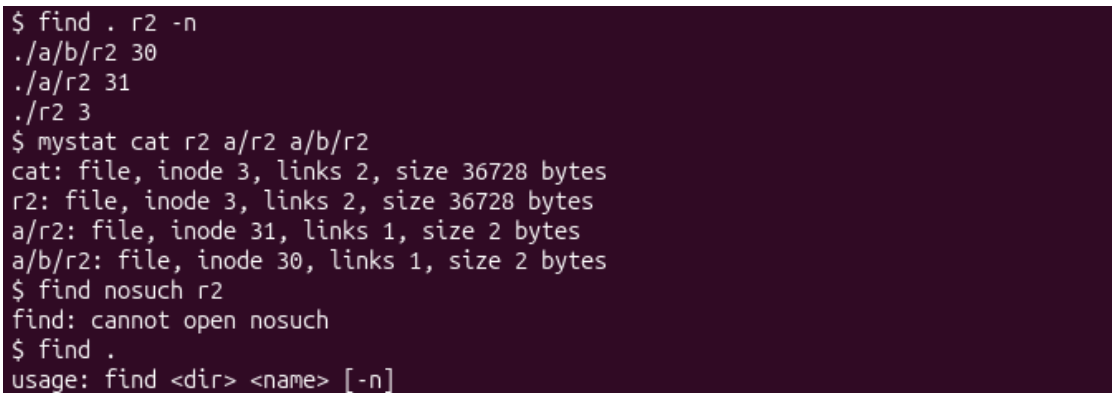


Figure 2.8 – Inode numbers with -n, and the error cases

CONCLUSIONS

The two laboratory works looked at the same structure from two sides. On Linux every observation pointed to the separation between names and inodes. A name is only an entry in a folder; the inode holds the type, the size, the permissions and the blocks. That is why a hard link is just a second entry with the same inode number, why rm frees nothing while another name remains, why a symbolic link, which stores a path instead of an inode, breaks when its target is removed, and why the link count of a folder grows with every subfolder. The permissions showed that the r and x bits mean different things on a folder, and the special bits explained how a normal user can change a password stored in a file that the user cannot even read.

The experiments also showed that the size of a file and the space it uses are different things, because space is given in whole blocks and a sparse file has no blocks at all, and that much of what looks like a file is not stored on a disk: /dev holds the devices, /proc is generated by the kernel when it is read, and a whole ext4 file system with 16384 inodes can live inside a file of 64 MB.

xv6 showed the same ideas in their smallest form. Its inode has only six fields, a directory entry is 16 bytes with a name of at most 14 characters, and a file can have at most 268 blocks. The program mystat made the hard link visible: the same inode under two names, the link count going up and down, and the data surviving the removal of the original name, exactly as on Linux. The program find showed that walking a tree needs only open, read, fstat and stat, and that the details matter: skipping the free slots and the entries . and .., ending names that fill all 14 bytes, and closing every folder. The differences found along the way, such as the root folder counting the .. of its subfolders but no folder counting its own ., were explained by reading a few lines of kernel/sysfile.c.

REFERENCES

1. The File System with Your Own Hands; Inside the xv6 File System. Laboratory Works 5 and 6, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.

2. COX, R., KAASHOEK, F., MORRIS, R. xv6: a simple, Unix-like teaching operating system. MIT, 2024 [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf

3. xv6 for RISC-V. Source code repository, MIT PDOS [online]. Available: https://github.com/mit-pdos/xv6-riscv

4. 6.1810 Lab: Xv6 and Unix utilities. MIT, 2024 [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/labs/util.html

5. inode(7), path_resolution(7), symlink(7). Linux manual pages [online]. Available: https://man7.org/linux/man-pages/

6. ext4 Data Structures and Algorithms. The Linux Kernel documentation [online]. Available: https://docs.kernel.org/filesystems/ext4/

7. RFC 1952. GZIP file format specification version 4.3, 1996 [online]. Available: https://www.rfc-editor.org/rfc/rfc1952